

FakeCraving — Product & Technical Requirements

Project folder name: `fakecraving` **One-line pitch:** A satirical “dopamine site” food-delivery app for India — users browse restaurants, add to cart, and place a fake order that is tracked in real time but never arrives. Inspired by Korea’s FoodNeverComes trend.

1. Project Overview

- No real food, no real payments, no real delivery.
 - The entire product is the *simulation* of ordering — browsing, cart, checkout, live tracking — followed by an honest reveal that nothing is coming.
 - Target audience: India, primarily mobile users, Gen Z / young professionals, shareable on social media (Instagram/X/Reels).
 - Success = virality via shares, not revenue. Optional donation button only.
-

2. Tech Stack

Layer	Choice
Frontend framework	React 19 + Tailwind CSS
Hosting	Vercel
Backend / DB	Supabase (PostgreSQL)
Analytics	Vercel Analytics
Type	Progressive Web App (PWA) — installable, responsive, NOT a native app
Auth	None — anonymous session ID stored in browser (localStorage/cookie)

No native iOS/Android build. No app store submission.

3. Folder Structure (suggested)

```
fakecraving/
├── public/
│   ├── manifest.json          # PWA manifest
│   ├── icons/                 # app icons for PWA (multiple sizes)
│   └── data/
│       └── restaurants.json    # merged Kaggle + synthetic data (seed)
├── src/
│   ├── components/
│   │   ├── layout/           # BottomNav (mobile), Sidebar (desktop), TopBar
│   │   ├── home/             # RestaurantCard, CuisineChip, Banner
│   │   ├── cart/              # CartItem, BillSummary, CheckoutButton
│   │   ├── tracking/          # MapMock, TrackingSteps, HonestyBox
│   │   ├── shared/            # DonateButton, LocationPicker, LanguageToggle,
│   │   └── share/              # ShareCard (generates shareable image)
│   ├── pages/ (or routes/)
│   │   ├── Home.jsx
│   │   ├── Cart.jsx
│   │   ├── Tracking.jsx
│   │   ├── OrderHistory.jsx
│   │   └── About.jsx          # "why this exists" page
│   ├── lib/
│   │   ├── supabaseClient.js
│   │   ├── geolocation.js     # geolocation + reverse geocoding
│   │   └── session.js         # anonymous session ID handling
│   ├── data/
│   │   └── seedMerge.py or .js # script to merge Kaggle + synthetic data (run o
│   ├── styles/
│   │   └── tokens.css          # design tokens (colors, fonts)
│   └── App.jsx
├── supabase/
│   └── schema.sql
└── README.md
```

4. Data Requirements

- **Source dataset:** Kaggle "Zomato Restaurants Dataset for Metropolitan Areas" (~123,657 rows, 13 Indian cities, columns include cuisine, city, price range, delivery info).
- **Do NOT use real restaurant names, logos, or exact addresses** — trademark/ToS risk.

- **Merge process:**

1. Keep from Kaggle: city, cuisine type, price range, approximate rating range.
2. Replace restaurant name field with AI-generated fictional names matching the cuisine style (e.g. "Rickshaw Biryani House" for Hyderabadi cuisine).
3. Generate synthetic dish-level data per restaurant (3–8 dishes each): name, price, short description, emoji/placeholder image.
4. Output as a static JSON file bundled with the app (no live restaurant API calls needed).

- This merge is a **one-time offline script**, not a runtime dependency.
-

5. Database Schema (Supabase / PostgreSQL)

```
-- Restaurants (seeded once from merged data)
create table restaurants (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  cuisine text,
  city text,
  price_range text,
  rating numeric,
  image_emoji text,
  created_at timestamptz default now()
);

-- Dishes
create table dishes (
  id uuid primary key default gen_random_uuid(),
  restaurant_id uuid references restaurants(id),
  name text not null,
  price numeric,
  description text,
  created_at timestamptz default now()
);

-- Orders (fake orders placed by users)
create table orders (
  id uuid primary key default gen_random_uuid(),
  session_id text not null,          -- anonymous, from browser
  restaurant_id uuid references restaurants(id),
  dish_ids uuid[],
```

```

    total_amount numeric,
    city text,
    created_at timestamptz default now()
);

-- Global stats (for live counter)
create table stats (
    key text primary key,
    value bigint default 0
);
-- seed: insert into stats (key, value) values ('total_orders', 0);

```

- No auth tables needed.
 - `stats.total_orders` incremented via a Postgres function/trigger whenever a row is inserted into `orders` (or incremented client-side via an RPC call).
-

6. Core Screens & Flow

1. Home / Browse

- Top bar: greeting + live location (area, city) + search bar
- Banner: satirical tagline ("Order anything. Pay nothing. Feel everything.")
- Cuisine filter chips
- Restaurant card list/grid (image, name, cuisine, rating, price, "FAKE · X min" badge)

2. Restaurant / Dish Detail (not yet mocked — needs building)

- Dish list with add-to-cart

3. Cart

- Item list with quantity controls
- "Fake order" reminder note
- Bill summary (item total, ₹0 delivery, ₹0 total)
- "Place fake order" button

4. Order Tracking

- Map mock with animated route + moving scooter icon

- ETA + status steps (Placed → Cooking → Picked up → Arriving)
- Honesty box reveal ("this order will never arrive")
- Share card generation prompt

5. **Order History** — list of past fake orders, reorder shortcut

6. **About / "Why this exists"** — short explainer page

7. Feature Requirements

Must-have (MVP)

- Live location detection (Geolocation API) → reverse geocode to area + city, with manual fallback if permission denied
- Responsive layout: mobile = bottom nav + single column; desktop = sidebar nav + grid
- Browse restaurants (filter by cuisine)
- Add to cart, adjust quantity
- Place fake order → tracking screen with animated map + status steps
- Honesty box message on tracking screen
- Global live order counter (from `stats` table)
- Donate button → UPI deep link (mobile) / UPI QR code (desktop)
- Dark mode toggle
- PWA setup (manifest, icons, installable)
- Vercel Analytics integration

Should-have (v1.1)

- Shareable order card (image generation) for social media — high priority for virality
- Reorder shortcut from order history
- Savings counter (₹ saved / calories not eaten) per order
- Fake satirical reviews/ratings on restaurant detail
- Hindi/English language toggle

- “Why this exists” info page

Nice-to-have (v1.2+)

- Random “sold out” / “restaurant closed” humor events
- Streak tracker (consecutive days of fake ordering)
- Fallback real recipe link per dish

Explicitly out of scope

- Real user accounts / login
 - Real payment integration
 - Native push notifications
 - Native iOS/Android apps
-

8. Design System

Colors

Token	Hex	Use
Marigold	#F2A93B	primary accent, banners
Aubergine	#3E1A45	headers, nav, CTAs
Chili Red	#C13F2E	ratings badges, checkout button
Curry Green	#4C7A4A	success states, rating tags
Ivory	#FFF8EC	background
Ink	#241A1D	text

Typography

- Display/headings: `Baloo 2` (rounded, playful, Bollywood-poster energy)
- Body: `Inter`

Signature UI element: the “honesty box” — a callout that breaks the illusion with a satirical,

warm message (e.g. "This order, like all your others here, will never arrive. That's the whole point.")

Layout patterns

- Mobile: bottom nav (Home / Search / Cart / Orders / Profile), single-column card list, phone-native feel
- Desktop: left sidebar nav, multi-column grid, browser-native feel

A working HTML/CSS reference mockup (Home, Cart, Tracking — mobile & desktop) has already been built and approved; use it as the visual source of truth for component styling.

9. Non-functional Requirements

- Fully responsive (mobile-first, breakpoints for tablet/desktop)
 - Fast load — static data bundle, minimal backend calls
 - HTTPS required (for Geolocation API) — default on Vercel
 - No cookie-consent-heavy analytics (prefer Vercel Analytics for simplicity)
 - Anonymous by design — no PII collected beyond an anonymous session ID
-

10. Open Items (to confirm before/during build)

- Final app name + domain (shortlisting in progress — `fakecraving.com` available on Namecheap, not yet purchased)
 - Final restaurant/dish image approach — emoji placeholders (as in mockup) vs. AI-generated illustration images
 - Reverse geocoding provider — Google Maps Geocoding API (paid, more accurate) vs. OpenStreetMap Nominatim (free, rate-limited)
-

11. Suggested Build Order

1. Scaffold React + Tailwind + Vercel project
2. Set up Supabase project + run `schema.sql`

3. Run offline data-merge script (Kaggle + synthetic) → produce `restaurants.json` seed
→ import into Supabase
4. Build layout shell (responsive nav: bottom nav / sidebar)
5. Build Home/Browse screen
6. Build Cart screen
7. Build Tracking screen (map mock, honesty box)
8. Add geolocation + reverse geocoding
9. Add donate button (UPI link + QR)
10. Add PWA manifest + icons
11. Add Vercel Analytics
12. Add v1.1 features (share card, reorder, savings counter, language toggle)